Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science, Semester V
Subject	CS3807 – Deep Learning Laboratory
Name	Madhav.S
Section	AIDS 'B'
Register Number	24011101066

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

---

1 Objective

The goal of this experiment was to implement a Multi-Layer Perceptron (MLP) using TensorFlow/Keras on the Fashion-MNIST dataset, covering image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2 Learning Outcomes

Through this experiment the following outcomes were achieved:

1. Explained the architecture of an MLP.
2. Preprocessed an image dataset for a dense-layer network.
3. Built and trained an MLP using Keras.
4. Evaluated classification performance using multiple metrics.
5. Performed automated hyperparameter optimization.
6. Recommended the best model configuration found by the search.

3 Background Theory

An MLP consists of an input layer, one or more hidden layers, and an output layer. Each layer l computes a weighted sum of its inputs, adds a bias, and passes the result through an activation function:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f\left(z^{(l)}\right) \tag{1}$$

The output layer uses a softmax activation to produce a probability distribution over the 10 classes:

$$\hat{y} = \text{Softmax}(z) \tag{2}$$

Training minimizes the categorical cross-entropy loss:

$$\mathcal{L} = -\sum_{i} y_i \log(\hat{y}_i) \tag{3}$$

The recommended architecture used for the baseline model in this experiment was:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4 Dataset

The Fashion-MNIST dataset was used for this experiment.

Table 1: Dataset Summary (verified at load time)

Property	**Value**
Training Images	60,000, shape $(60000, 28, 28)$
Testing Images	10,000, shape $(10000, 28, 28)$
Classes	10
Image Size	28×28
Pixel Range	0 to 255

The 10 classes are: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

5 Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector, so each image was flattened:

$$28 \times 28 = 784 \tag{4}$$

The following preprocessing steps were performed:

1. Loaded Fashion-MNIST.
2. Displayed sample images (Figure 1).
3. Flattened images from $(N, 28, 28)$ to $(N, 784)$.
4. Normalized pixel values from $[0, 255]$ to $[0, 1]$.
5. Converted labels into one-hot vectors.

Table 2: Tensor Shapes Before and After Preprocessing

Tensor	**Before**	**After**
Training Images	$(60000, 28, 28)$	$(60000, 784)$
Testing Images	$(10000, 28, 28)$	$(10000, 784)$
Training Labels	$(60000,)$	$(60000, 10)$ one-hot
Testing Labels	$(10000,)$	$(10000, 10)$ one-hot

6 Experimental Procedure

Task 1: Dataset Exploration

The dataset dimensions were printed and ten sample images were displayed (Figure 1), along with the class distribution across the training set (Figure 2).

Task 2: Data Preprocessing

Images were flattened and normalized as described in Section 5, with tensor shapes printed before and after each step (Table 2).

Task 3: Model Construction

The baseline MLP was constructed with the recommended architecture:
Input(784) → Dense(128, ReLU) → Dense(64, ReLU) → Dense(10, Softmax).

Table 3: Baseline Model Summary

Layer	**Output Shape**	**Parameters**
Dense (ReLU)	(None, 128)	100,480
Dense_1 (ReLU)	(None, 64)	8,256
Dense_2 (Softmax)	(None, 10)	650
Total Parameters		109,386
Trainable Parameters		109,386
Non-trainable Parameters		0

Task 4: Model Training

The baseline model was compiled with the Adam optimizer (learning rate 0.001), categorical cross-entropy loss, and accuracy as the evaluation metric. It was trained for 20 epochs with a batch size of 32, using a 10% validation split.

Table 4: Baseline Training Log (Selected Epochs)

Epoch	**Train Acc.**	**Train Loss**	**Val. Acc.**	**Val. Loss**
1	0.8189	0.5036	0.8470	0.4107
5	0.8907	0.2924	0.8813	0.3264
10	0.9105	0.2377	0.8835	0.3246
15	0.9230	0.2032	0.8907	0.3336
20	0.9337	0.1748	0.8847	0.3890

Task 5: Model Evaluation

The trained baseline model was evaluated on the held-out test set.

Table 5: Baseline Model Test Performance

Metric	**Value**
Accuracy	0.8836
Precision (macro)	0.8853
Recall (macro)	0.8836
F1-score (macro)	0.8839

Table 6: Baseline Model – Per-Class Classification Report

Class	Precision	Recall	F1-score
T-shirt/top	0.84	0.81	0.83
Trouser	0.99	0.97	0.98
Pullover	0.76	0.84	0.80
Dress	0.91	0.89	0.90
Coat	0.84	0.75	0.79
Sandal	0.94	0.98	0.96
Shirt	0.69	0.72	0.70
Sneaker	0.96	0.94	0.95
Bag	0.96	0.98	0.97
Ankle boot	0.96	0.94	0.95
Macro avg	0.89	0.88	0.88
Weighted avg	0.89	0.88	0.88

7 Hyperparameter Optimization

Instead of manually selecting hyperparameters, automated hyperparameter optimization was performed using `RandomizedSearchCV` together with the SciKeras wrapper (`KerasClassifier`), with 5-fold cross-validation.

Table 7: Hyperparameter Search Space

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

The search was run on a stratified 10,000-sample subsample of the training set (15 candidates $\times$ 5 folds = 75 model fits) to keep the search computationally feasible, with the final optimized model retrained on the full 60,000-image training set using the best hyperparameters found.

8 Mandatory Plots

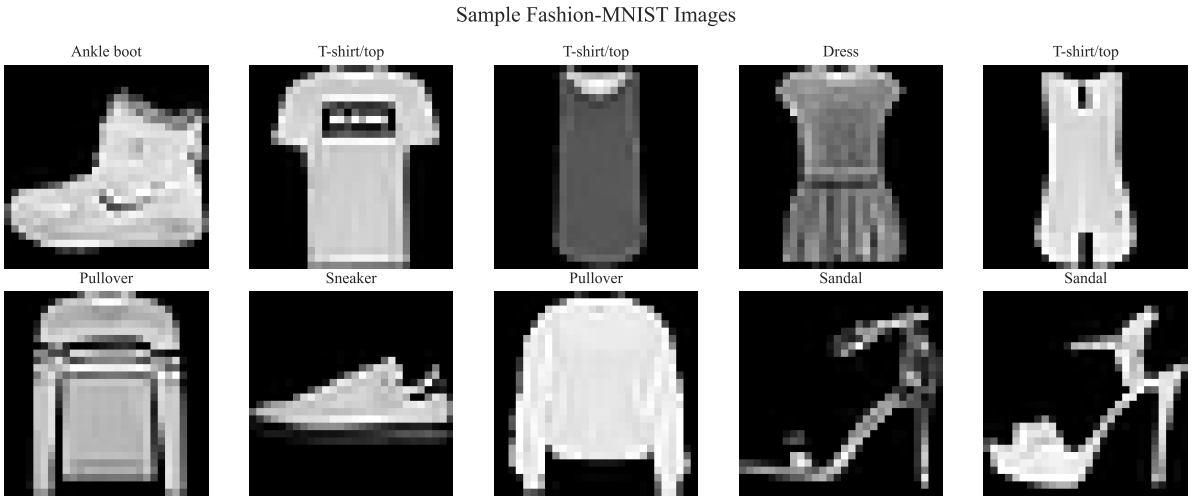

Figure 1: Sample Fashion-MNIST Images

Inference: Each of the ten classes is visually distinct even at 28×28 resolution, though some categories (e.g. Shirt, T-shirt/top, Pullover, and Coat) share similar silhouettes.

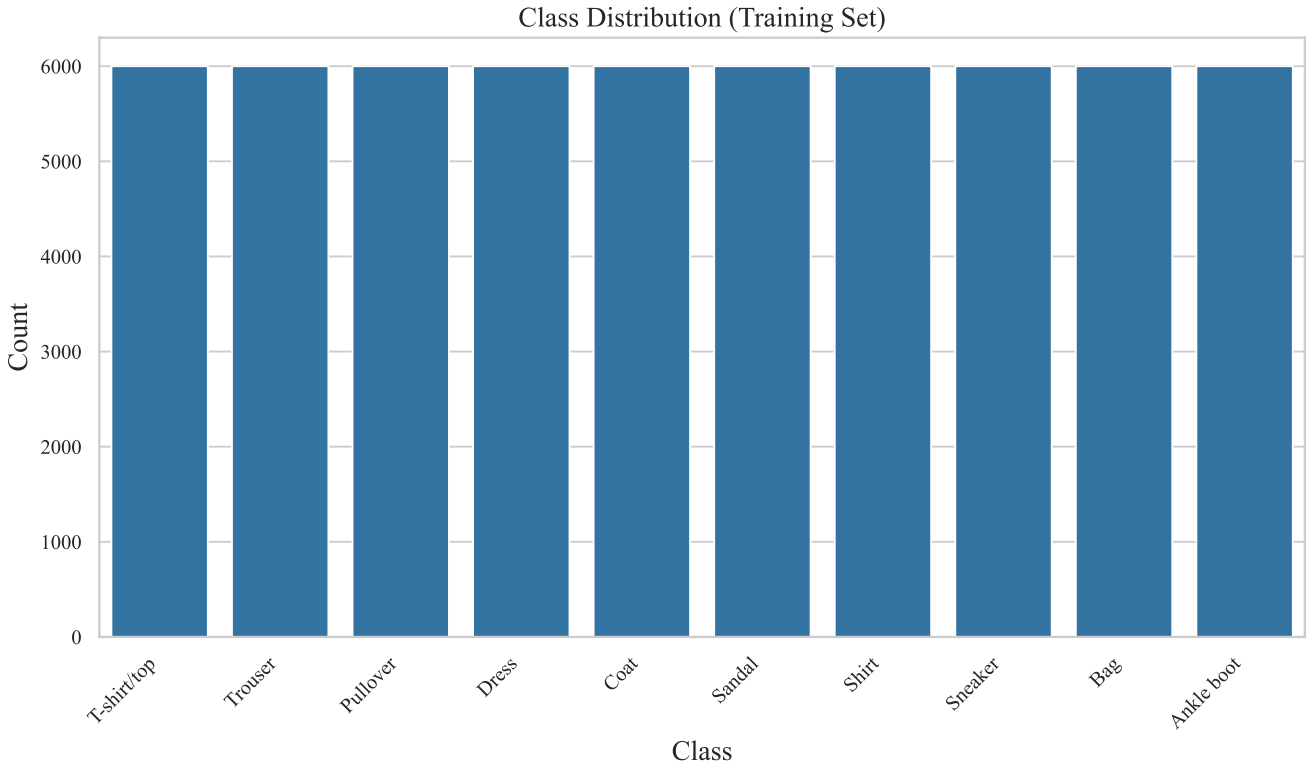

Figure 2: Class Distribution (Training Set)

Inference: All 10 classes are represented by exactly 6,000 training images each, so the dataset is perfectly balanced. This means accuracy is a reliable metric here and no class-imbalance handling was required.

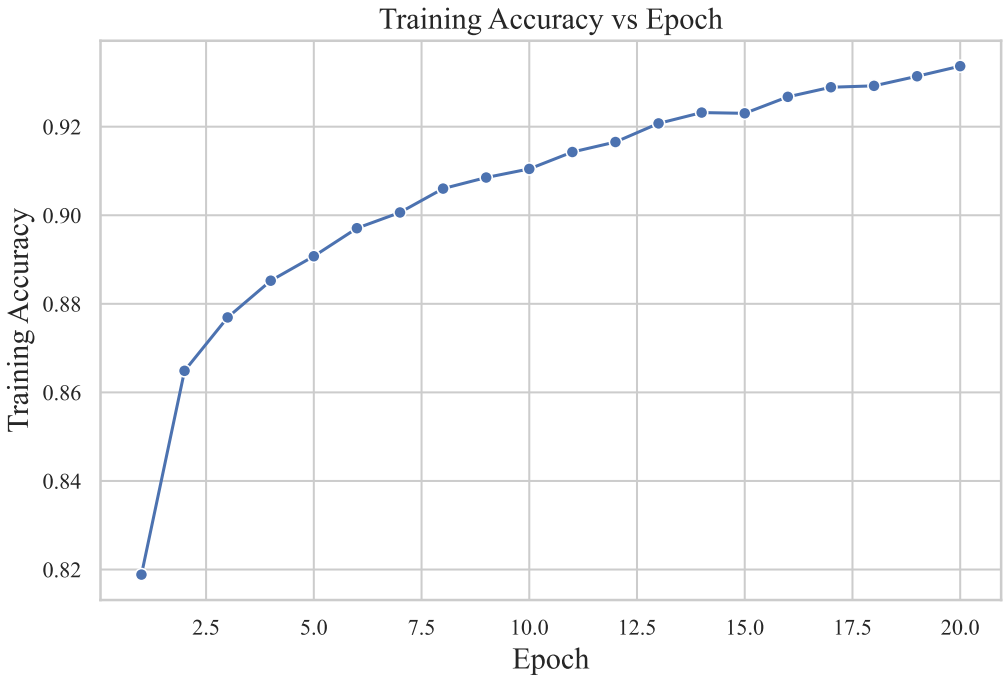

Figure 3: Training Accuracy vs Epoch

Inference: Training accuracy increases steadily and monotonically from 81.9% to 93.4% over 20 epochs, showing the model continues to fit that the training data gets better at every epoch.

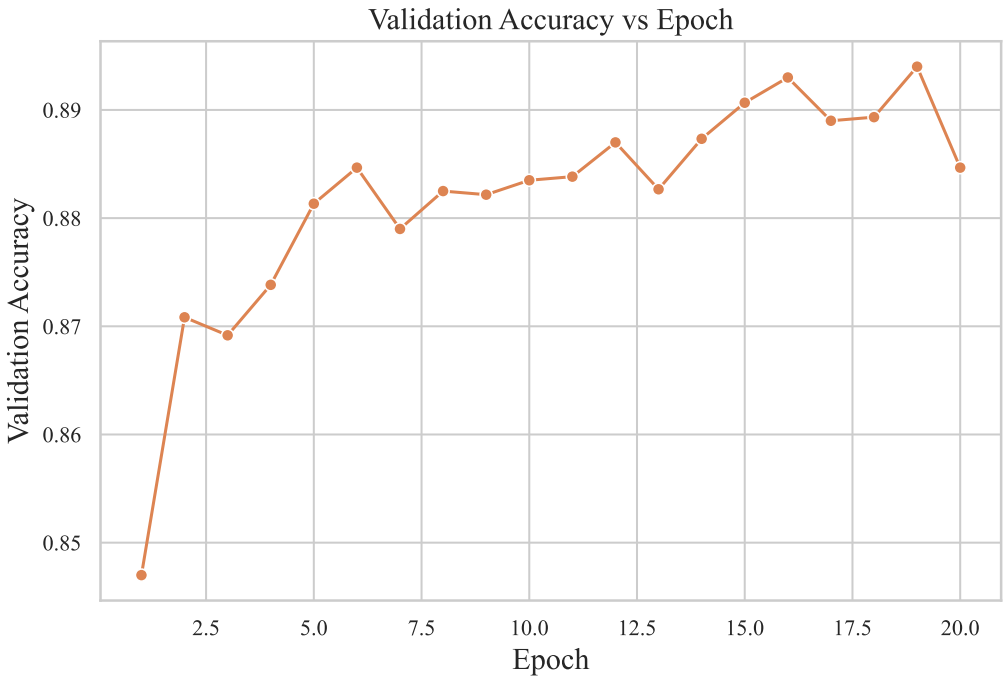

Figure 4: Validation Accuracy vs Epoch

Inference: Validation accuracy rises quickly in the first few epochs and then plateaus around 88–89%, well below the training accuracy of 93%+ by epoch 20. This shows that the model begins to overfit on the training set in the later epochs.

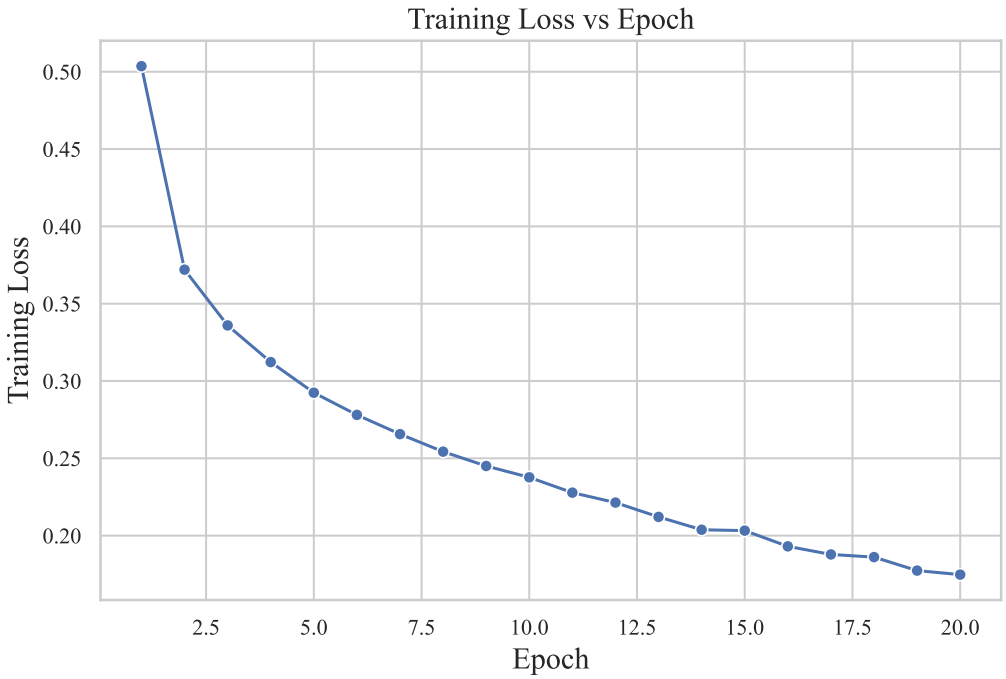

Figure 5: Training Loss vs Epoch

Inference: Training loss decreases smoothly and consistently from 0.50 to 0.17 across all 20 epochs, almost similar to the training accuracy curve. This confirms stable optimization with the Adam optimizer.

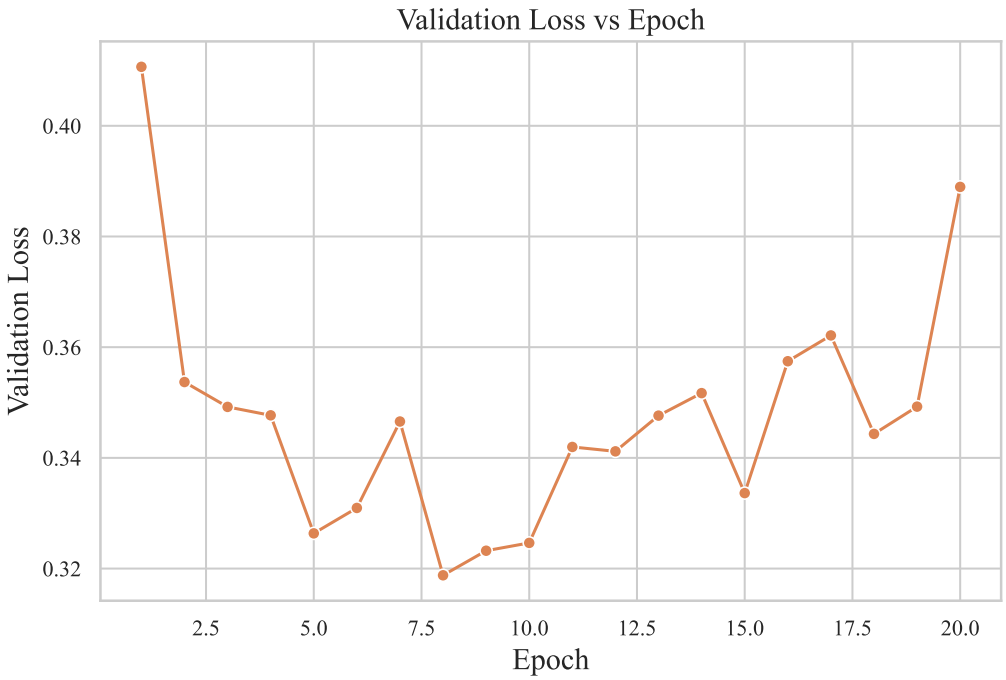

Figure 6: Validation Loss vs Epoch

Inference: Validation loss decreases until around epoch 10–15 and then begins to increase(ending near 0.39), while training loss keeps decreasing. This is the classic divergence pattern that confirms overfitting in the later epochs.

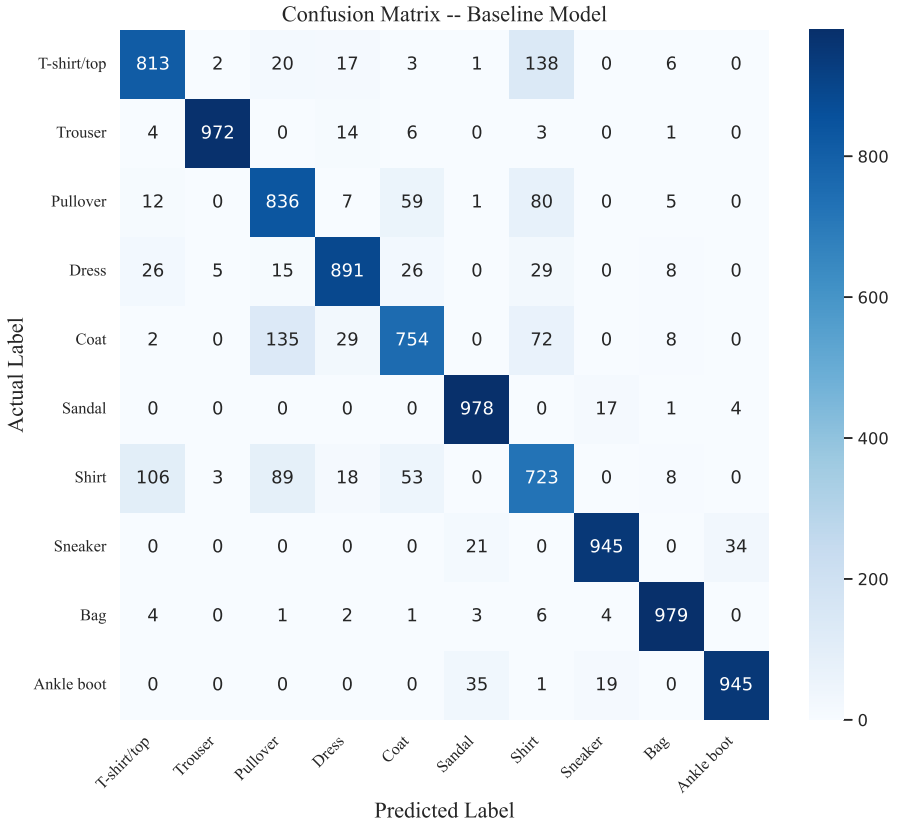

Figure 7: Confusion Matrix – Baseline Model

Inference: Most confusion occurs between Shirt, T-shirt/top, Pullover, and Coat – all upper-body garments with similar silhouettes at low resolution. Trouser, Sandal, Bag, and Ankle boot are classified almost perfectly, consistent with their more unique shapes.

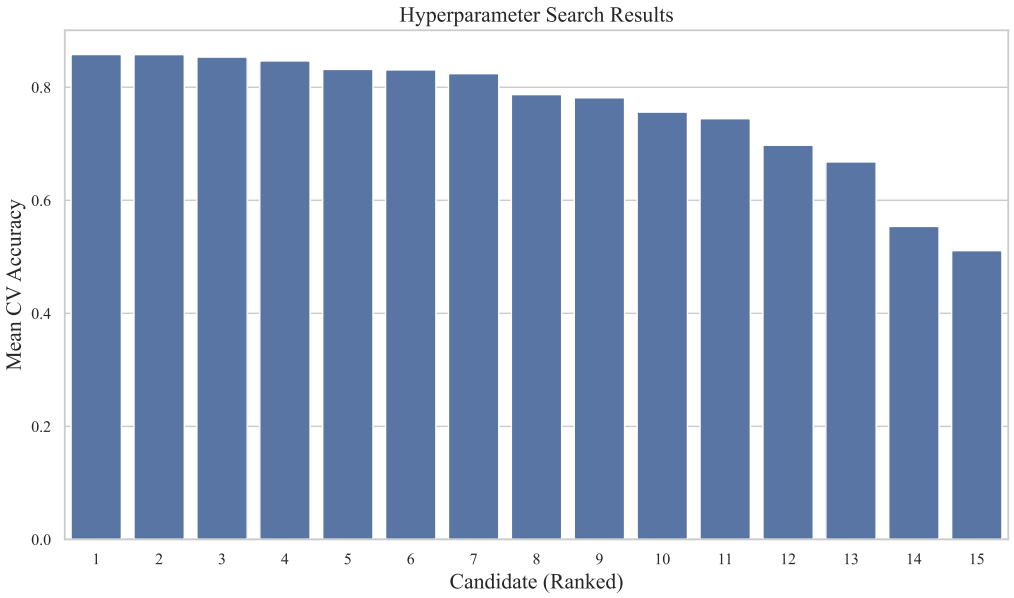

Figure 8: Hyperparameter Search Results

Inference: Cross-validation accuracy varies noticeably across the 15 sampled candidates, ranging from weak configurations up to the best candidate at 85.79% mean CV accuracy, showing

that the hyperparameter choice has a real, measurable effect on the model quality even on a subsampled training set.

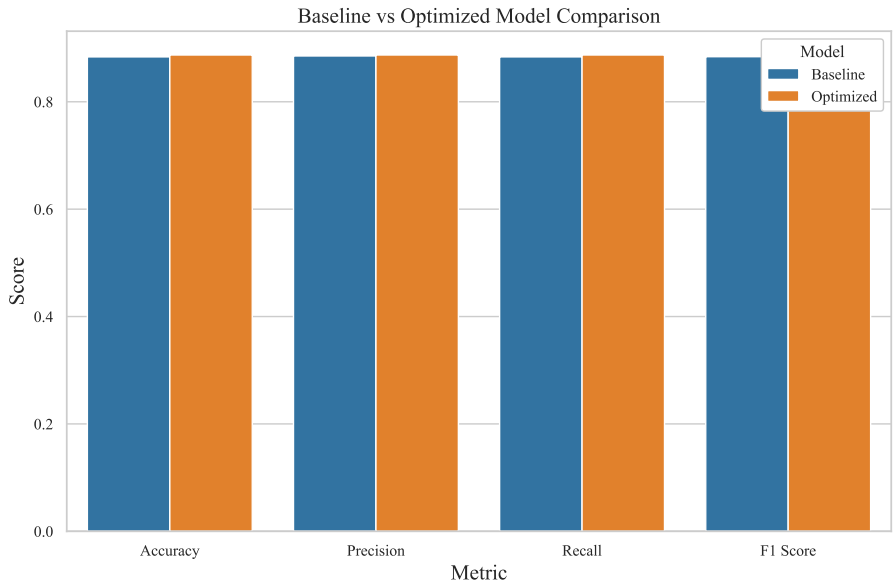

Figure 9: Best Model Accuracy Comparison

Inference: The optimized model outperforms the baseline on every metric (accuracy, precision, recall, and F1-score), though the margin is slight (under 0.4 percentage points), indicating that the baseline architecture was already a reasonably strong choice for this dataset.

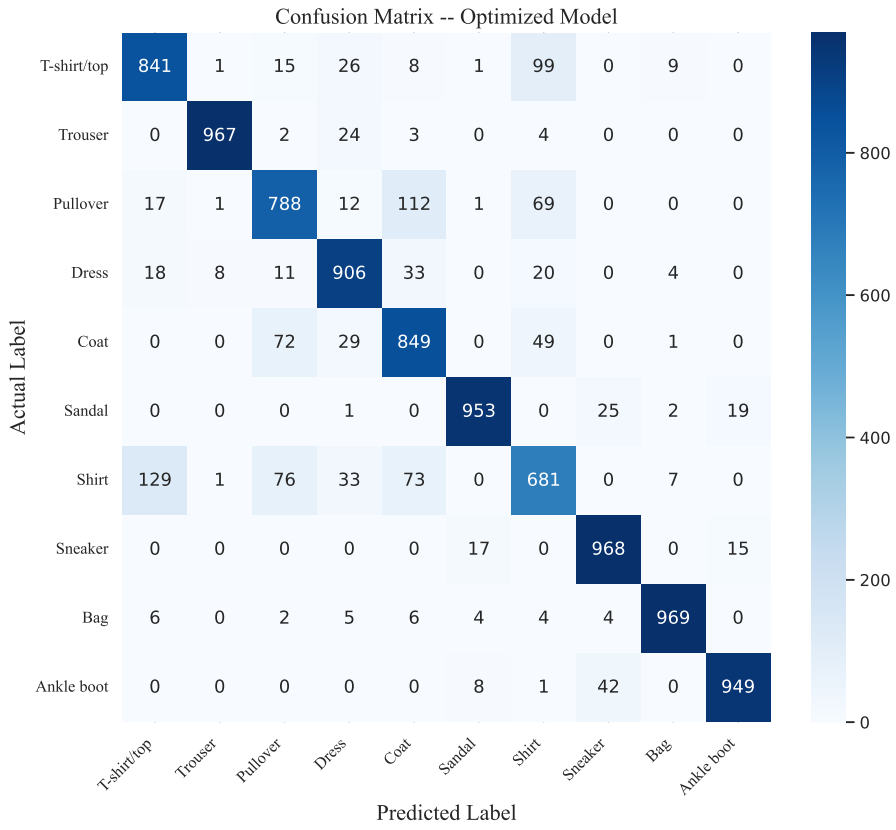

Figure 10: Confusion Matrix – Optimized Model (bonus)

Inference: The optimized model shows the same overall confusion pattern as the baseline (Shirt/T-shirt/Pullover/Coat remain the hardest to separate), but with slightly fewer total misclassifications, consistent with its marginally higher test accuracy.

9 Source Code

The complete implementation, exactly as run in Colab, is hosted on GitHub.
https://github.com/smadhav2024/Deep_Learning_Lab/tree/main/Lab%202
The repository contains the full notebook `deep_learning_lab_2.ipynb`, the `requirements.txt` needed to run it, the raw `.eps` plots, and this report's Latex code.

10 Results

Table 8: Best Hyperparameters Found (RandomizedSearchCV, 5-fold CV)

Hyperparameter	**Best Value**
Hidden Layers	1
Hidden Neurons	128
Learning Rate	0.001
Batch Size	128
Optimizer	Adam
Activation Function	Sigmoid
Epochs	30
Dropout	0.0
Cross-validation Accuracy	0.8579
Testing Accuracy (after full retrain)	0.8871

Table 9: Performance Comparison: Baseline vs Optimized

Metric	**Baseline**	**Optimized**
Accuracy	0.8836	0.8871
Precision	0.8853	0.8871
Recall	0.8836	0.8871
F1-score	0.8839	0.8868

11 Additional Task (K4): Perceptron Learning Algorithm for the XOR Gate

Objective

The perceptron learning algorithm was implemented from scratch in Python for the XOR logic gate. The weights were logged after every update, the decision boundary was plotted after each weight update, and the pattern that prevents the single-layer perceptron from converging on XOR was analyzed.

Implementation

A single-layer perceptron with two input weights and a bias was trained on the XOR truth table using the standard perceptron update rule:

$$w \leftarrow w + \eta\,(t - \hat{y})\,x, \qquad b \leftarrow b + \eta\,(t - \hat{y}) \tag{5}$$

where $\hat{y} = \text{step}(w \cdot x + b)$, t is the target output, and $\eta = 0.1$ is the learning rate. Weights and bias were initialized to zero and the perceptron was trained for 5 epochs over the 4 XOR samples $\{(0,0)\to 0,\ (0,1)\to 1,\ (1,0)\to 1,\ (1,1)\to 0\}$, with a weight update logged and the weight/bias state recorded every time a sample was misclassified.

Weights After Each Update

Table 10: Perceptron Weight Updates on the XOR Gate ($\eta = 0.1$, initial $w = [0, 0]$, $b = 0$)

Update	**Epoch**	**Input**	**Target**	**Predicted**	w_1	w_2	**Bias**
1	1	(0,0)	0	1	0.0000	0.0000	−0.1000
2	1	(0,1)	1	0	0.0000	0.1000	0.0000
3	1	(1,1)	0	1	−0.1000	0.1000	−0.1000
4	2	(0,1)	1	0	−0.1000	0.1000	0.0000
5	2	(1,0)	1	0	0.0000	0.1000	0.1000
6	2	(1,1)	0	1	−0.1000	0.1000	0.0000
7	3	(0,0)	0	1	−0.1000	0.1000	−0.1000
8	3	(0,1)	1	0	−0.1000	0.1000	0.0000
9	3	(1,0)	1	0	0.0000	0.1000	0.1000
10	3	(1,1)	0	1	−0.1000	0.1000	0.0000
11	4	(0,0)	0	1	−0.1000	0.1000	−0.1000
12	4	(0,1)	1	0	−0.1000	0.1000	0.0000
13	4	(1,0)	1	0	0.0000	0.1000	0.1000
14	4	(1,1)	0	1	−0.1000	0.1000	0.0000
15	5	(0,0)	0	1	−0.1000	0.1000	−0.1000
16	5	(0,1)	1	0	−0.1000	0.1000	0.0000
17	5	(1,0)	1	0	0.0000	0.1000	0.1000
18	5	(1,1)	0	1	−0.1000	0.1000	0.0000

Note that from Update 4 onward the weight/bias state simply repeats the same 4-update cycle $(-0.1, 0.1, 0.0) \to (0.0, 0.1, 0.1) \to (-0.1, 0.1, 0.0) \to (-0.1, 0.1, -0.1) \to \ldots$ every epoch, and the number of misclassifications per epoch was [3, 3, 4, 4, 4] over the 5 epochs – it never drops to 0, i.e. the perceptron never converges.

Decision Boundary Evolution

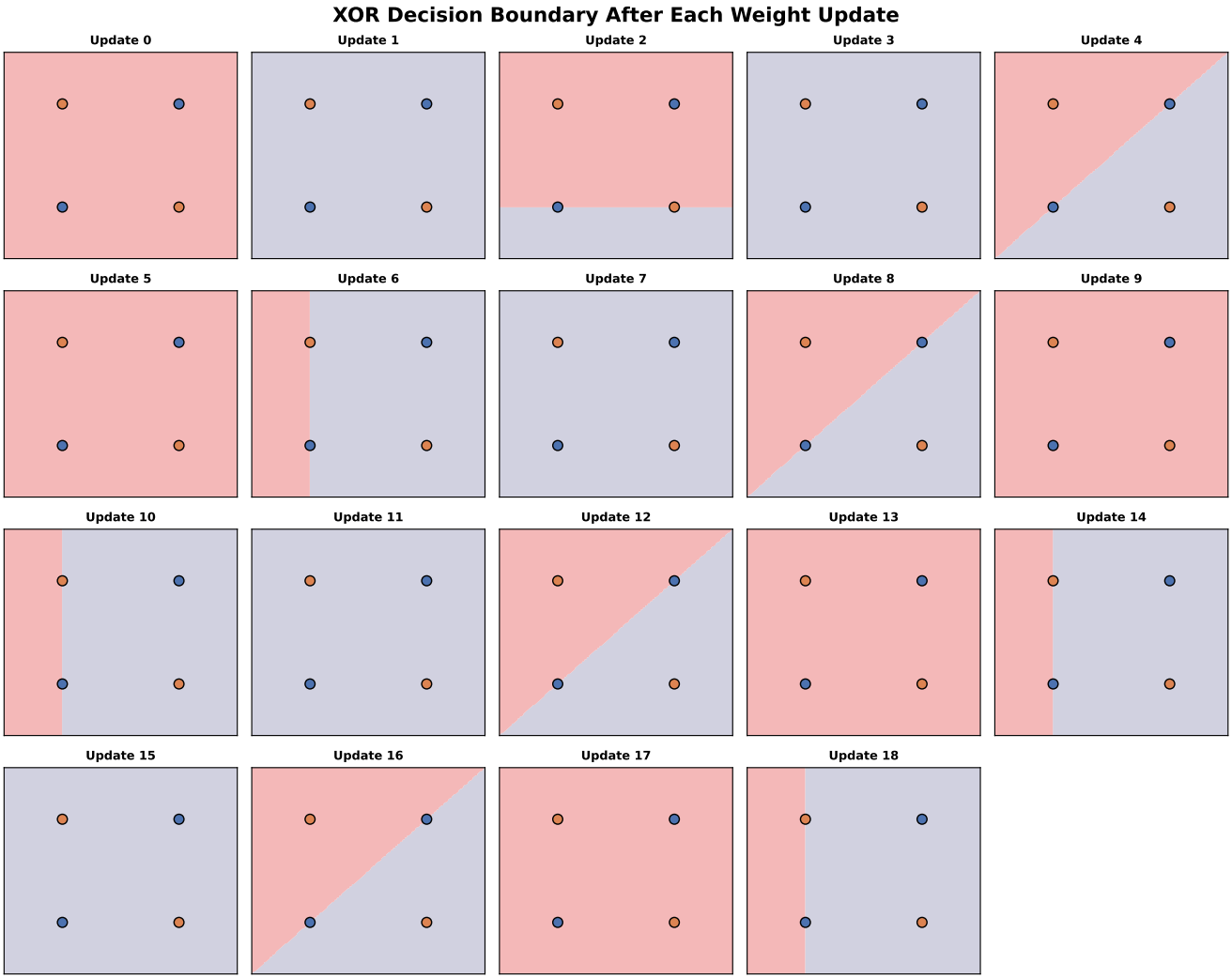

Figure 11: XOR Decision Boundary After Each of the 18 Weight Updates (Update 0 = initial state)

Inference: The decision boundary line visibly rotates and shifts after every update, moving to correct whichever point was just misclassified. However, no single straight-line boundary manages to place all the four XOR points on their correct sides simultaneously. The boundary keeps going after the errors from one update to the next without ever settling.

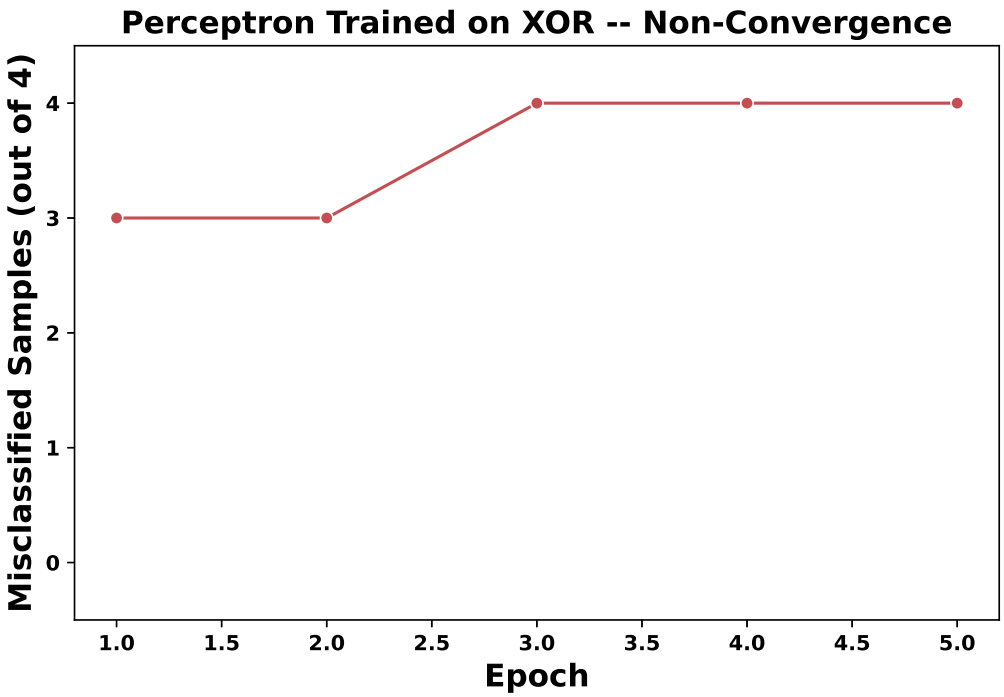

Figure 12: Perceptron Trained on XOR – Non-Convergence

Inference: The number of misclassified samples per epoch $(3, 3, 4, 4, 4)$ never reaches zero and does not trend downward – after epoch 2 it stabilizes at 4 out of 4 samples misclassified at every epoch. This confirms that the perceptron has entered a permanent error cycle rather than converging.

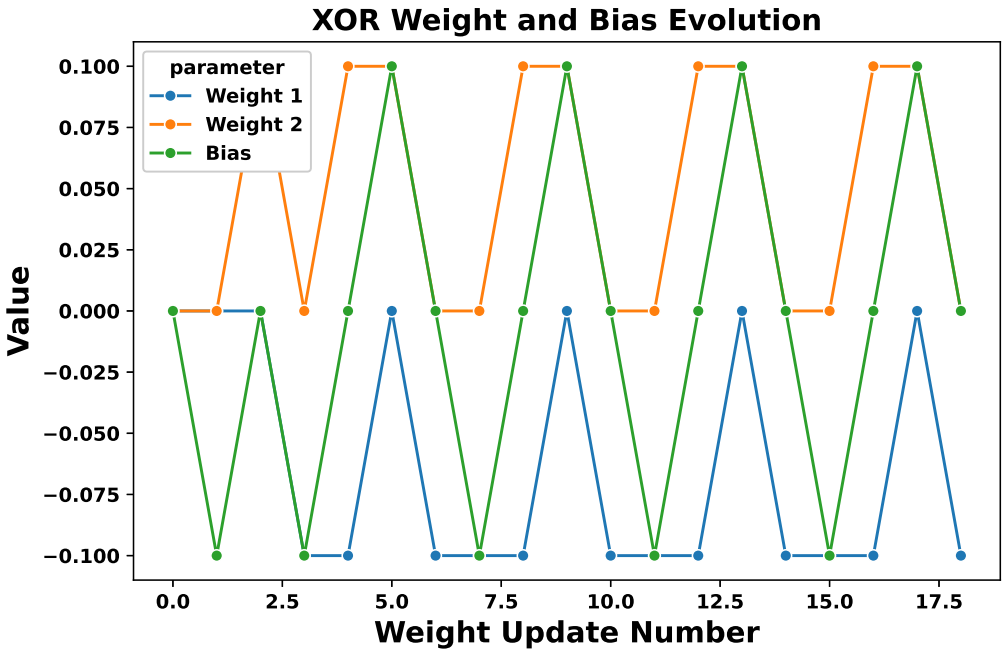

Figure 13: XOR Weight and Bias Evolution Over Updates

Inference: After the initial updates (Updates 1–3), both weights and the bias settle into a repeating oscillation between a small set of fixed values instead of converging to a stable point. The direct numerical evidence of the cyclical, non-convergent behaviour can be seen in Figure 12.

Analysis: Why the Perceptron Never Converges on XOR

The pattern responsible for this non-convergence is that **XOR is not linearly separable**. The two positive samples $(0, 1)$ and $(1, 0)$ and the two negative samples $(0, 0)$ and $(1, 1)$ cannot be separated by any single straight line in the 2D input space – they form two pairs of diagonally opposite corners of the unit square, so any line that correctly separates one pair of opposite corners necessarily misclassifies the other pair. Every weight update in Figure 11 rotates or shifts the boundary to fix whichever point was just misclassified, but doing so always breaks a different point that was previously correct, since no orientation of a single straight line can keep all four points on their correct sides at once. This is exactly why solving XOR requires at least one hidden layer, combining multiple linear boundaries produces a non-linear decision region that a single-layer perceptron, limited to one straight-line boundary, can never represent.

12 Discussion

1. **Which optimization method was used?**
   RandomizedSearchV with 5-fold cross-validation was used, sampling 15 candidate hyperparameter combinations from the full search space rather than searching and evaluating all of them.

2. **Which hyperparameters were selected?**
   The best configuration found was: 1 hidden layer of 128 neurons, sigmoid activation, no dropout, the Adam optimizer with a learning rate of 0.001, a batch size of 128, and 30 training epochs.

3. **Did optimization improve performance?**
   Yes, though slightly: test accuracy improved from 0.8836 (baseline) to 0.8871 (optimized), with corresponding small gains in precision, recall, and F1-score. However, the optimized model used a simpler architecture(a single hidden layer) than the two-hidden-layer baseline, yet still performs slightly better, showing that for this dataset, tuning the learning rate, batch size, and training length mattered more than adding architectural depth.

4. **Which hyperparameter had the greatest impact?**
   Based on the spread seen in the hyperparameter search results plot (Figure 8), the learning rate and optimizer choice had the most visible impact on cross-validation accuracy – configurations using a learning rate of 0.1 tended to perform considerably worse, likely due to overshooting weight updates, while learning rates of 0.001–0.01 paired with Adam or RMSProp consistently gave stronger results.

5. **Compare Grid Search and Randomized Search.**
   Grid Search exhaustively evaluates every combination in the search space, guaranteeing the best combination within the grid is found, but its cost grows eponentially with the number of hyperparameters and their candidate values which is infeasible here, since the full grid ($3\times4\times3\times4\times3\times3\times3\times3 = 34{,}992$ combinations) would require tens of thousands of model fits. Randomized Search instead samples a fixed number of combinations (15, here) from the same space,which completes in a practical amount of time, which is why it was the recommended and chosen method for this experiment.

6. **Recommend the best MLP configuration.**
   For this dataset and task, the recommended configuration is the one found by the search: a single hidden layer of 128 neurons with sigmoid activation, no dropout, trained with Adam (learning rate 0.001), batch size 128, for 30 epochs. This configuration achieved the highest test accuracy (0.8871) while also being computationally lighter than the two-hidden-layer baseline.

13 Report Contents

This report includes the Objective, Background Theory, Dataset description, Experimental Procedure (Tasks 1–5), Hyperparameter Optimization methodology and search space, all mandatory Plots with inference, Results (best hyperparameters and performance comparison), the Additional Task (K4: Perceptron Learning Algorithm for the XOR Gate), Discussion, and References.

14 References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. Fashion-MNIST Dataset, Zalando Research.
5. TensorFlow/Keras Documentation.
6. SciKeras Documentation.